

Diseño y Documentación Formal de API REST mediante la Especificación OpenAPI 3.0 en un Microservicio de Gestión de Productos

Farinango Guandinango, Freddy Vladimir; Gaibor Rodríguez, Jeremy Ruperto; Villamarín Cuenca, Iván Andrés

Asignatura: Aplicaciones Distribuidas — Equipo C — 02 de junio de 2026

Abstract

Resumen—La ausencia de contratos formales en el desarrollo de servicios REST constituye un obstáculo recurrente en proyectos basados en arquitecturas de microservicios, dado que incrementa la ambigüedad en la integración entre componentes y dificulta el mantenimiento evolutivo de los sistemas. El presente trabajo documenta la práctica de diseño y documentación formal del microservicio principal del proyecto TiendaTech, denominado API REST de Productos, mediante la especificación OpenAPI 3.0. El objetivo central consistió en generar un contrato de servicio completo, validado y exportable que sirviera como artefacto de referencia para el equipo de desarrollo y como base para futuras integraciones en el Proyecto Formativo de Curso. La metodología empleada comprendió la identificación sistemática de los endpoints funcionales del microservicio, el diseño incremental del contrato en Swagger Editor, la definición de esquemas de datos reutilizables, la configuración del mecanismo de autenticación Bearer JWT y la validación sintáctica y semántica del contrato resultante. Como resultados concretos se obtuvo la documentación de cinco endpoints REST con sus respectivos parámetros, cuerpos de solicitud y respuestas HTTP tipificadas; la definición de esquemas para entidades centrales del dominio; la incorporación del esquema de seguridad JWT al contrato; la exportación del artefacto en formato YAML; y la generación de una colección Postman funcional. Se concluye que la especificación OpenAPI 3.0 contribuye de manera efectiva a la estandarización de contratos de servicio en arquitecturas distribuidas, reduciendo la ambigüedad comunicacional entre equipos y proporcionando una base sólida para la interoperabilidad y el mantenimiento sostenido del sistema.

Palabras clave: OpenAPI 3.0, API REST, microservicios, documentación de servicios, Swagger, JWT, sistemas distribuidos.

Abstract

Abstract—The absence of formal contracts in REST service development constitutes a recurring obstacle in projects based on microservice architectures, as it increases ambiguity in component integration and complicates the evolutionary maintenance of systems. This paper documents the practice of formal design and documentation of the main microservice of the TiendaTech project, referred to as the Products REST API, using the OpenAPI 3.0 specification. The central objective was to produce a complete, validated, and exportable service contract to serve as a reference artifact for the development team and as a foundation for future integrations within the Course Formative Project. The methodology included the systematic identification of the functional endpoints of the microservice, the incremental design of the contract in Swagger Editor, the definition of reusable data schemas, the configuration of the Bearer JWT authentication mechanism, and the syntactic and semantic validation of the resulting contract. Concrete results include the documentation of five REST endpoints with their respective parameters, request bodies, and typed HTTP responses; the definition of schemas for core domain entities; the incorporation of the JWT security scheme into the contract; the export of the artifact in YAML format; and the generation of a functional Postman collection. It is concluded that the OpenAPI 3.0 specification effectively contributes to the standardization of service contracts in distributed architectures, reducing communicational ambiguity between teams and providing a solid foundation for interoperability and sustained system maintenance.

Keywords: OpenAPI 3.0, REST API, microservices, service documentation, Swagger, JWT, distributed systems.

I. INTRODUCCIÓN

El desarrollo de aplicaciones distribuidas ha experimentado una transformación significativa durante la última década, impulsada por la adopción generalizada de arquitecturas basadas en microservicios. En este paradigma, cada componente funcional del sistema se expone como un servicio independiente accesible a través de interfaces de programación de aplicaciones (API) que utilizan el protocolo HTTP como mecanismo de transporte. Las API REST, fundamentadas en los principios definidos por Fielding [1], se han consolidado como el estándar predominante para la comunicación entre microservicios debido a su simplicidad, escalabilidad y compatibilidad con infraestructuras heterogéneas.

Sin embargo, la proliferación de servicios en una arquitectura distribuida introduce desafíos considerables en términos de documentación y gobernanza de las interfaces. Cuando la

documentación de una API se produce de manera informal, ya sea mediante wikis internas, comentarios en el código fuente o acuerdos verbales entre equipos, surgen problemas recurrentes que comprometen la calidad del software. Entre estos problemas se identifican la desincronización entre la implementación real del servicio y su especificación documentada, la ambigüedad en los contratos de integración y la dificultad para incorporar nuevos desarrolladores al proyecto. Según Haupt et al. [2], la falta de documentación formal constituye uno de los principales factores que incrementan los costos de mantenimiento en sistemas orientados a servicios.

Ante esta problemática, la especificación OpenAPI, anteriormente conocida como Swagger Specification, surge como una respuesta estandarizada de la industria. OpenAPI 3.0 define un formato neutral al lenguaje de programación para describir APIs RESTful de forma legible tanto por máquinas

como por seres humanos, permitiendo la generación automática de documentación interactiva, la validación de contratos y la creación de clientes y servidores a partir de una especificación única [3]. Esta capacidad lo posiciona como una herramienta fundamental dentro de la filosofía *API First*, en la cual el contrato formal del servicio constituye el artefacto inicial del proceso de desarrollo, previo a la implementación.

El proyecto TiendaTech, desarrollado en el contexto de la asignatura Aplicaciones Distribuidas, implementa un sistema de comercio electrónico basado en microservicios. El microservicio de productos representa el componente central del sistema, exponiéndose a través de endpoints REST que permiten la consulta, búsqueda y categorización del catálogo de productos. La ausencia de un contrato formal para este servicio representaba un riesgo latente para las etapas de integración previstas en el Proyecto Formativo de Curso, lo que motivó la ejecución de la presente práctica.

A partir del contexto descrito, la pregunta que guía la presente investigación es: ¿Cómo contribuye la especificación formal mediante OpenAPI 3.0 a la estandarización, documentación y mantenibilidad de servicios REST en arquitecturas basadas en microservicios?

II. FUNDAMENTACIÓN TEÓRICA

A. Arquitectura de Microservicios y Sistemas Distribuidos

La arquitectura de microservicios representa un estilo arquitectónico en el cual una aplicación se estructura como un conjunto de servicios pequeños e independientes, cada uno ejecutándose en su propio proceso y comunicándose mediante mecanismos ligeros. Este enfoque contrasta con la arquitectura monolítica tradicional, en la que todos los componentes del sistema residen en un único proceso desplegable. Newman [4] señala que los microservicios permiten el despliegue independiente de cada componente, la escalabilidad granular y la elección de tecnologías heterogéneas por equipo, lo que resulta en una mayor agilidad organizacional.

Un sistema distribuido, en términos más generales, es aquel en el cual los componentes de cómputo se encuentran ubicados en nodos de red interconectados y se coordinan mediante el intercambio de mensajes. Los sistemas distribuidos modernos enfrentan desafíos inherentes como la consistencia eventual, la tolerancia a particiones de red y la latencia de comunicación, descritos formalmente en el teorema CAP. En el contexto de los microservicios, la comunicación entre servicios se realiza predominantemente mediante API REST síncronas o sistemas de mensajería asíncronos como Apache Kafka. La correcta documentación de las interfaces de comunicación resulta crítica para garantizar la cohesión del sistema.

B. APIs REST y Principios de la Arquitectura REST

El estilo arquitectónico REST fue definido formalmente por Fielding en su disertación doctoral del año 2000 [1]. Sus principios fundamentales incluyen la interfaz uniforme, la ausencia de estado en el servidor (*statelessness*), el uso de caché, la arquitectura cliente-servidor, la capacidad de construir sistemas en capas y la transferencia de código bajo demanda

como restricción opcional. La interfaz uniforme, considerada el principio central de REST, establece que los recursos deben ser identificados mediante URI, que la representación del recurso en el mensaje es independiente de su estado interno en el servidor, que los mensajes deben ser autodescriptivos y que se debe utilizar el concepto de hipermedia como motor del estado de la aplicación (HATEOAS).

En la práctica, las implementaciones de REST utilizan los métodos HTTP como mecanismo semántico para expresar la intención de la operación. El método GET se emplea para la recuperación de recursos sin efectos secundarios; POST para la creación de nuevos recursos o la ejecución de operaciones complejas; PUT y PATCH para la actualización total y parcial respectivamente; y DELETE para la eliminación. La API REST del proyecto TiendaTech hace uso de los métodos GET y POST para las operaciones de consulta y búsqueda de productos, respetando la semántica HTTP en cada caso.

C. OpenAPI 3.0 y Swagger

La especificación OpenAPI, actualmente en su versión 3.1, constituye el estándar de facto para la descripción formal de APIs RESTful [3]. Su versión 3.0, adoptada en la presente práctica, introduce mejoras sustanciales respecto a la versión 2.0 —anteriormente conocida como Swagger Specification— entre las que destacan el soporte para múltiples servidores, la separación del objeto *Components* para la reutilización de esquemas, y una mayor capacidad de expresión para la definición de tipos de medios y esquemas de seguridad.

Un documento OpenAPI 3.0 se estructura en torno a objetos principales: *info*, que contiene los metadatos del API; *servers*, que especifica las URLs base del servicio; *paths*, que describe los endpoints disponibles y sus operaciones; *components*, que centraliza los esquemas reutilizables, parámetros y respuestas; y *security*, que define los mecanismos de autenticación aplicables globalmente o por operación. Esta estructura facilita la modularización de la especificación y promueve la reutilización de definiciones a lo largo del documento.

Swagger Editor es la herramienta oficial de edición de contratos OpenAPI proporcionada por SmartBear. Su interfaz basada en navegador permite la edición simultánea del YAML y la previsualización en tiempo real de la documentación generada, así como la detección inmediata de errores sintácticos y semánticos. La combinación de Swagger Editor con la especificación OpenAPI constituye el entorno estándar para la implementación del paradigma *API First*.

D. Contratos API First y Documentación Formal

El enfoque *API First* propone que el contrato formal del servicio debe ser el primer artefacto producido en el ciclo de desarrollo, actuando como un acuerdo vinculante entre proveedores y consumidores del servicio antes de que exista implementación alguna. Según Serbout et al. [5], este paradigma reduce significativamente los defectos de integración al establecer expectativas claras y verificables desde las etapas iniciales del proyecto. En contraste, el enfoque *Code First*, en el cual la documentación se genera a partir del código existente, propende a producir documentación desactualizada o incompleta

cuando el ritmo de cambio del código supera la cadencia de actualización de la documentación.

La documentación formal de APIs a través de contratos OpenAPI ofrece ventajas adicionales más allá de la comunicación entre equipos. Entre ellas se encuentran la posibilidad de generar automáticamente stubs de cliente y servidor a partir de la especificación, la facilidad para escribir pruebas de contrato automatizadas y la capacidad de integrar la validación del contrato en pipelines de integración continua. Estas características convierten al contrato OpenAPI en un artefacto vivo que evoluciona junto con el sistema y actúa como fuente única de verdad sobre la interfaz del servicio.

E. Formatos YAML y JSON

OpenAPI 3.0 admite la expresión del contrato tanto en formato JSON como en YAML. El formato YAML (*YAML Ain't Markup Language*) es preferido por los desarrolladores debido a su mayor legibilidad gracias a la indentación significativa y la ausencia de delimitadores como llaves y corchetes. YAML es un superconjunto de JSON, lo que garantiza la interoperabilidad entre ambos formatos a través de herramientas de conversión estándar. La estructura jerárquica de un contrato OpenAPI se expresa de manera natural en YAML, con cada nivel de indentación representando un nivel de anidamiento en el árbol de objetos de la especificación.

F. Postman y Pruebas de APIs

Postman es una plataforma de colaboración para el desarrollo de APIs que permite la creación, organización y ejecución de solicitudes HTTP de manera visual. Sus colecciones agrupan las solicitudes relacionadas con un mismo servicio o conjunto de operaciones, facilitando su ejecución individual o automatizada. En el contexto del presente trabajo, Postman fue utilizado para organizar los cinco endpoints documentados, configurar los encabezados de autenticación Bearer JWT y verificar el comportamiento del servicio mediante la inspección de las respuestas HTTP recibidas.

G. Seguridad en APIs REST y JWT

La seguridad en APIs REST constituye un aspecto crítico en el diseño de sistemas distribuidos. Entre los mecanismos más utilizados se encuentran la autenticación mediante API Keys, OAuth 2.0 y JSON Web Tokens (JWT). Bertocci [6] describe JWT como un estándar abierto (RFC 7519) que define un formato compacto y autocontenido para la transmisión segura de información entre partes mediante objetos JSON firmados digitalmente. Un token JWT se compone de tres partes codificadas en Base64URL separadas por puntos: el encabezado (*header*), que especifica el algoritmo de firma; el *payload*, que contiene las afirmaciones (*claims*) sobre el usuario y el contexto de la sesión; y la firma (*signature*), que garantiza la integridad del token.

En el contexto de OpenAPI 3.0, el esquema de seguridad Bearer JWT se configura dentro del objeto `securitySchemes` del componente *components*, con un tipo HTTP y un esquema *bearer*. Su aplicación puede ser global, afectando a todas las operaciones del API,

o local, restringiéndose a operaciones específicas. Esta granularidad permite expresar con precisión qué endpoints requieren autenticación y cuáles son de acceso público, un aspecto relevante para microservicios con niveles de acceso diferenciados.

H. Interoperabilidad, Versionamiento y Buenas Prácticas

La interoperabilidad entre servicios en arquitecturas distribuidas depende en gran medida de la estabilidad y precisión de los contratos de interfaz. Un contrato OpenAPI bien definido actúa como una garantía de que los cambios en la implementación interna del servicio no romperán a los consumidores existentes, siempre que la interfaz pública permanezca compatible. El versionamiento de APIs es una práctica recomendada que permite evolucionar el servicio introduciendo cambios incompatibles en una nueva versión sin afectar a los consumidores de versiones anteriores. Estrategias comunes incluyen el versionamiento en la URL (p.ej., `/api/v1/productos`), en el encabezado HTTP o mediante negociación de contenido. Haupt et al. [2] documentan que el versionamiento semántico aplicado a contratos OpenAPI facilita la detección automatizada de cambios incompatibles en pipelines de CI/CD.

III. MATERIALES Y MÉTODOS

A. Entorno Tecnológico

La práctica se desarrolló utilizando un conjunto de herramientas ampliamente adoptadas en la industria para el diseño y documentación de APIs. Swagger Editor, en su versión web disponible en `editor.swagger.io`, fue empleado como entorno principal de edición y validación del contrato OpenAPI 3.0. Esta herramienta ofrece retroalimentación inmediata sobre errores sintácticos y semánticos, así como una vista previa interactiva de la documentación generada en formato Swagger UI.

Postman fue utilizado en su versión de escritorio para la creación de la colección de pruebas y la verificación de la conectividad con los endpoints documentados. GitHub sirvió como repositorio centralizado para el versionamiento de los artefactos generados, organizados bajo la carpeta `/docs/api/` del proyecto TiendaTech. El microservicio objeto de documentación fue el servicio de productos de TiendaTech, implementado con Spring Boot y expuesto en el puerto local 8080 durante las sesiones de prueba.

B. Procedimiento

Paso 1. Identificación del microservicio principal. El equipo realizó un análisis del proyecto TiendaTech para identificar el microservicio de mayor relevancia funcional. Se determinó que el servicio de productos (*productos-service*) constituía el componente central del sistema, dado que expone la lógica de negocio principal relacionada con el catálogo de la tienda.

Paso 2. Selección de endpoints. A partir del análisis del código fuente del controlador REST del microservicio, se identificaron cinco endpoints funcionales: el listado paginado de productos, la consulta por identificador, la consulta por categoría, la búsqueda por texto libre y la recuperación del catálogo de

categorías. Estos endpoints representan la superficie pública del servicio consumida por los demás componentes del sistema.

Paso 3. Diseño del contrato OpenAPI. Se inició la redacción del contrato en Swagger Editor comenzando por el objeto *info*, en el cual se especificó el título del API (TiendaTech - API de Productos), la versión (1.0.0) y una descripción detallada del servicio. Se configuró el objeto *servers* con la URL base del microservicio.

Paso 4. Definición de esquemas. En el objeto *components/schemas* se definieron los esquemas de datos correspondientes a las entidades principales del dominio: *ProductoDTO*, *ProductoPageResponse*, *CategoriaDTO* y *BusquedaRequest*. El uso de esquemas reutilizables mediante referencias (*\$ref*) garantizó la consistencia de las definiciones a lo largo del contrato.

Paso 5. Configuración de autenticación Bearer JWT. En el objeto *components/securitySchemes* se definió el esquema de seguridad denominado *bearerAuth*, con tipo *http* y esquema *bearer*, especificando el formato *bearerFormat* como JWT. Este esquema fue referenciado globalmente mediante el objeto *security* a nivel de documento.

Paso 6. Documentación de endpoints. Para cada uno de los cinco endpoints identificados se documentaron los parámetros de entrada (path, query y body según corresponda), los posibles códigos de respuesta HTTP (200, 400, 404 y 500) y los esquemas asociados a cada respuesta.

Paso 7. Validación en Swagger Editor. El contrato completo fue validado en Swagger Editor, verificando la ausencia de errores sintácticos y advertencias semánticas. Se corrigieron inconsistencias detectadas durante la validación hasta obtener un contrato válido según la especificación OpenAPI 3.0.

Paso 8. Exportación YAML. Una vez validado el contrato, se procedió a su exportación en formato YAML mediante la función de descarga de Swagger Editor. El archivo resultante, denominado *tiendatech-api.yaml*, fue almacenado en la carpeta */docs/api/* del repositorio del proyecto.

Paso 9. Creación de la colección Postman. Se creó manualmente en Postman una colección denominada *TiendaTech API* con cinco carpetas organizadas por recurso. Se ejecutaron las solicitudes contra el microservicio en ejecución local para verificar la coherencia entre el contrato documentado y el comportamiento real del servicio.

Paso 10. Versionamiento en GitHub. Los artefactos generados (contrato YAML y colección Postman en formato JSON) fueron confirmados en el repositorio GitHub del proyecto mediante commits descriptivos.

IV. RESULTADOS

La ejecución de los diez pasos metodológicos descritos en la sección anterior produjo un conjunto de artefactos verificables y reproducibles. Los resultados se presentan a continuación organizados por categoría.

A. Endpoints Documentados

Se documentaron cinco endpoints REST correspondientes a las operaciones de consulta y búsqueda del microservicio de

productos de TiendaTech. La Tabla 1 sintetiza los endpoints documentados.

Table 1: Endpoints REST documentados en el contrato OpenAPI 3.0

Nº	Endpoint	Método	Descripción
1	GET /api/productos	GET	Listado paginado
2	GET /api/productos/{id}	GET	Consulta por ID
3	GET /api/productos/por-categ.	GET	Filtrado por categoría
4	POST /api/productos/buscar	POST	Búsqueda por texto
5	GET /api/categorias	GET	Consulta de categorías

El endpoint `GET /api/productos` admite los parámetros de consulta *page* y *size* para la paginación de resultados, retornando un objeto *ProductoPageResponse* que encapsula la lista de productos junto con metadatos de paginación. El endpoint `POST /api/productos/buscar` recibe un cuerpo de solicitud de tipo *BusquedaRequest* con un campo *query* de texto libre, cuyo procesamiento es delegado a la capa de servicio del microservicio.

B. Respuestas HTTP Configuradas

Para cada endpoint se configuraron respuestas asociadas a cuatro códigos de estado HTTP. La Tabla 2 detalla los códigos documentados.

Table 2: Códigos de respuesta HTTP documentados por operación

Código	Nombre	Uso en el contrato
200	OK	Respuesta exitosa con payload
400	Bad Request	Parámetros inválidos o cuerpo malformado
404	Not Found	Recurso no encontrado
500	Internal Server Error	Error inesperado del servidor

C. Esquemas de Datos Definidos

En el objeto *components/schemas* del contrato se definieron cinco esquemas reutilizables. La Tabla 3 describe cada esquema definido.

Table 3: Esquemas de datos definidos en components/schemas

Esquema	Descripción
ProductoDTO	Representación de un producto con id, nombre, descripción, precio y categoría
ProductoPageResponse	Respuesta paginada (content, totalPages, totalElements)
CategoriaDTO	Representación de una categoría con id y nombre
BusquedaRequest	Cuerpo de solicitud para búsqueda por texto libre (campo: query)
ErrorResponse	Estructura estándar de error con código, mensaje y timestamp

D. Autenticación Bearer JWT

El contrato incorpora un esquema de seguridad de tipo HTTP denominado *bearerAuth*, configurado con el esquema *bearer* y el formato *bearerFormat*: `JWT`. Este esquema fue aplicado globalmente a nivel de documento, lo que indica que todos los endpoints requieren un token JWT válido en el encabezado *Authorization* de la solicitud. La configuración del

esquema en el contrato permite que Swagger UI muestre automáticamente un campo de ingreso de token en la interfaz interactiva de documentación.

E. Artefactos Generados

Como resultado final del proceso, se generaron dos artefactos almacenados en la carpeta `/docs/api/` del repositorio GitHub del proyecto TiendaTech. La Tabla 4 describe los artefactos producidos.

Table 4: Artefactos generados durante la práctica

Artefacto	Descripción
<code>tiendatech-api.yaml</code>	Contrato OpenAPI 3.0 validado y exportado
<code>TiendaTech.postman_collection.json</code>	Colección Postman con 5 endpoints configurados

El contrato YAML fue validado exitosamente en Swagger Editor sin errores ni advertencias. La colección Postman permitió verificar la conectividad con el microservicio en ejecución local y confirmar que las respuestas obtenidas eran coherentes con los esquemas definidos en el contrato.

V. DISCUSIÓN

Los resultados obtenidos en la presente práctica son consistentes con las ventajas reportadas en la literatura científica sobre la adopción de contratos OpenAPI en entornos de microservicios. La generación de un contrato formal validado para el microservicio de productos de TiendaTech resolvió la ambigüedad preexistente sobre la interfaz del servicio, estableciendo un punto de referencia único para todos los consumidores del API.

Serbout et al. [5] documentan en un estudio empírico sobre repositorios públicos de GitHub que la calidad de los contratos OpenAPI varía considerablemente en función de la completitud de los esquemas definidos y la cobertura de los códigos de respuesta. En el presente trabajo, la decisión de definir esquemas para todas las entidades del dominio y de documentar cuatro códigos de respuesta por operación se alinea con las prácticas de alta calidad identificadas en dicho estudio, contribuyendo a la mantenibilidad del contrato a medida que el sistema evolucione.

Respecto a la autenticación, la incorporación del esquema Bearer JWT en el contrato OpenAPI proporciona una ventaja adicional: la generación automática de clientes que incluyan el encabezado de autenticación correspondiente. Esto elimina la necesidad de documentar por separado el mecanismo de autenticación en wikis o guías de integración, reduciendo el riesgo de desincronización entre la documentación formal y la implementación real, un problema identificado por Haupt et al. [2] como prevalente en proyectos de software de mediana y gran escala.

La organización de los artefactos bajo el directorio `/docs/api/` del repositorio GitHub constituye una buena práctica que facilita la localización de la documentación por parte de nuevos miembros del equipo y habilita la integración de la validación del contrato en pipelines de CI/CD. Merelo-

Guervós et al. [7] argumentan que la trazabilidad de los artefactos de documentación en sistemas de control de versiones es un indicador de madurez del proceso de desarrollo de software, asociado positivamente con métricas de calidad del código.

A. Limitaciones

La primera limitación identificada reside en el alcance de la práctica, que cubrió únicamente los endpoints de lectura del microservicio de productos. Las operaciones de escritura (creación, actualización y eliminación de productos), presentes en la implementación real del servicio, no fueron documentadas en este ciclo de trabajo, lo que resulta en un contrato parcial que no cubre la totalidad de la superficie del API.

La segunda limitación se relaciona con la ausencia de ejemplos concretos en los objetos de solicitud y respuesta del contrato. Si bien se definieron los esquemas de datos, la inclusión de valores de ejemplo mediante la propiedad *example* o *examples* de OpenAPI 3.0 habría enriquecido la utilidad de la documentación interactiva y facilitado la comprensión del comportamiento esperado del servicio por parte de los desarrolladores consumidores.

B. Amenazas a la Validez

Se identificaron tres amenazas principales a la validez de los resultados. En primer lugar, la validez interna del contrato está supeditada a que los esquemas definidos reflejen con exactitud los tipos de datos producidos y consumidos por la implementación actual del microservicio; cualquier discrepancia no detectada durante la validación en Swagger Editor podría manifestarse como errores en tiempo de ejecución durante las pruebas de integración. En segundo lugar, la validez externa de los resultados es limitada dado que la práctica se circunscribió a un único microservicio de un proyecto académico, por lo que las conclusiones extraídas no pueden generalizarse directamente a sistemas productivos de mayor escala y complejidad sin validación adicional. En tercer lugar, la validez de constructo podría verse afectada si la herramienta Swagger Editor presentara diferencias en el comportamiento del validador respecto a otras implementaciones de la especificación OpenAPI 3.0, como el validador integrado en frameworks como Springdoc o Connexion.

C. Líneas de Trabajo Futuro

Como primera línea de trabajo futuro se propone la extensión del contrato para cubrir la totalidad de los endpoints del microservicio de productos, incluyendo las operaciones de escritura protegidas por roles de autorización diferenciados. La implementación de pruebas de contrato automatizadas mediante herramientas como Dredd o Schemathesis, que validan la conformidad de las respuestas del servidor con los esquemas definidos en el contrato OpenAPI, constituiría una mejora sustancial en la garantía de calidad del sistema. Como segunda línea, se propone la generación automática de clientes tipados para los lenguajes utilizados por los equipos consumidores del API, aprovechando las capacidades del generador OpenAPI Generator, lo que reduciría el tiempo

de integración y minimizaría los errores derivados de la codificación manual de llamadas HTTP.

VI. CONCLUSIONES

La especificación formal del microservicio de productos de TiendaTech mediante OpenAPI 3.0 demostró ser un mecanismo efectivo para la estandarización de la interfaz del servicio en el contexto de una arquitectura basada en microservicios. El proceso metodológico aplicado produjo un contrato YAML validado que documentó con precisión los cinco endpoints funcionales del servicio, sus parámetros de entrada, los esquemas de datos del dominio y el mecanismo de autenticación Bearer JWT, constituyendo así una respuesta verificable a la pregunta de investigación planteada.

La adopción de OpenAPI 3.0 como instrumento de documentación contribuyó directamente a la reducción de la ambigüedad en la interfaz del servicio al establecer un contrato formal que actuó como fuente única de verdad para todos los componentes del sistema que interactúan con el microservicio de productos. Esta propiedad resultó especialmente relevante en el contexto del Proyecto Formativo de Curso, donde la integración entre múltiples microservicios desarrollados por equipos distintos requería acuerdos precisos sobre los contratos de interfaz.

La herramienta Swagger Editor facilitó la detección temprana de inconsistencias en el contrato durante su elaboración, eliminando defectos que de otro modo habrían emergido en etapas más costosas del ciclo de desarrollo. La organización de los artefactos resultantes en el repositorio GitHub bajo una estructura de directorio estandarizada sentó las bases para la evolución controlada del contrato y su eventual integración en flujos de validación automatizados.

En conclusión, los resultados de la práctica confirmaron que la especificación OpenAPI 3.0 contribuyó a la estandarización del contrato de servicio, a la mejora de la documentación técnica del microservicio y a la preparación del sistema para su integración futura con otros componentes de la arquitectura, elementos que en conjunto favorecen la mantenibilidad sostenida del proyecto TiendaTech.

REFERENCES

- [1] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, Dept. Inform. Comput. Sci., Univ. California Irvine, Irvine, CA, USA, 2000. [Online]. Available: <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>
- [2] M. Haupt, C. Leymann, and C. Pautasso, "A conversation about versioning RESTful APIs," in *Proc. IEEE Int. Conf. Web Serv. (ICWS)*, Chicago, IL, USA, 2021, pp. 398–407. doi: 10.1109/ICWS53863.2021.00058
- [3] D. Miller, J. Broeckelmann, and K. Schreiber, "OpenAPI 3.0: Evolution of the API specification," *IEEE Softw.*, vol. 37, no. 3, pp. 22–28, May/Jun. 2020. doi: 10.1109/MS.2020.2975921
- [4] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2021.
- [5] S. Serbout, C. Pautasso, U. Zdun, and O. Zimmermann, "From OpenAPI fragments to API pattern primitives and design smells," in *Proc. 16th Eur. Conf. Pattern Lang. Programs (EuroPLoP)*, Irsee, Germany, 2021, pp. 1–12. doi: 10.1145/3489449.3489998
- [6] V. Bertocci, "JSON Web Token (JWT) profile for OAuth 2.0 access tokens," IETF RFC 9068, Oct. 2021. doi: 10.17487/RFC9068
- [7] J. J. Merelo-Guervós, P. García-Sánchez, and M. G. Arenas, "Ci-droid: Understanding code quality through GitHub actions and version control," *SoftwareX*, vol. 17, p. 100962, Jan. 2022. doi: 10.1016/j.softx.2022.100962
- [8] E. Wilde and E. Wilde, "RESTful API design for distributed systems," *IEEE Internet Comput.*, vol. 24, no. 6, pp. 31–39, Nov./Dec. 2020. doi: 10.1109/MIC.2020.3020118
- [9] A. Alarcon-Aquino, O. Starostenko, and J. M. Ramirez-Cortes, "Automated REST API testing and documentation using machine learning approaches," *J. Syst. Softw.*, vol. 196, p. 111535, Feb. 2023. doi: 10.1016/j.jss.2022.111535
- [10] C. Pautasso, O. Wilde, and R. Alarcon, "REST: Advanced Research Topics and Practical Applications," in *Proc. Int. World Wide Web Conf. Companion*, Lyon, France, 2020. doi: 10.1007/978-1-4614-9299-3

I. FRAGMENTO REPRESENTATIVO DEL CONTRATO OPENAPI 3.0

A continuación se presenta el fragmento del objeto *info* y la configuración del esquema de seguridad del contrato OpenAPI 3.0 generado durante la práctica. El contrato completo se encuentra disponible en el repositorio GitHub del proyecto bajo la ruta `/docs/api/tiendatech-api.yaml`.

```
openapi: "3.0.3"

info:
  title: TiendaTech - API de Productos
  version: "1.0.0"
  description: API REST para la gestion del catalogo de productos

components:
  securitySchemes:
    bearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT
```

II. FRAGMENTO DE DEFINICIÓN DE ENDPOINT

```
paths:
  /api/productos/{id}:
    get:
      summary: Obtener producto por ID
      operationId: getProductoById
      security:
        - bearerAuth: []
      parameters:
        - name: id
          in: path
          required: true
          schema:
            type: integer
```

III. EVIDENCIAS DE LA PRÁCTICA

Las siguientes figuras corresponden a capturas de pantalla que documentan los resultados visuales de la práctica.

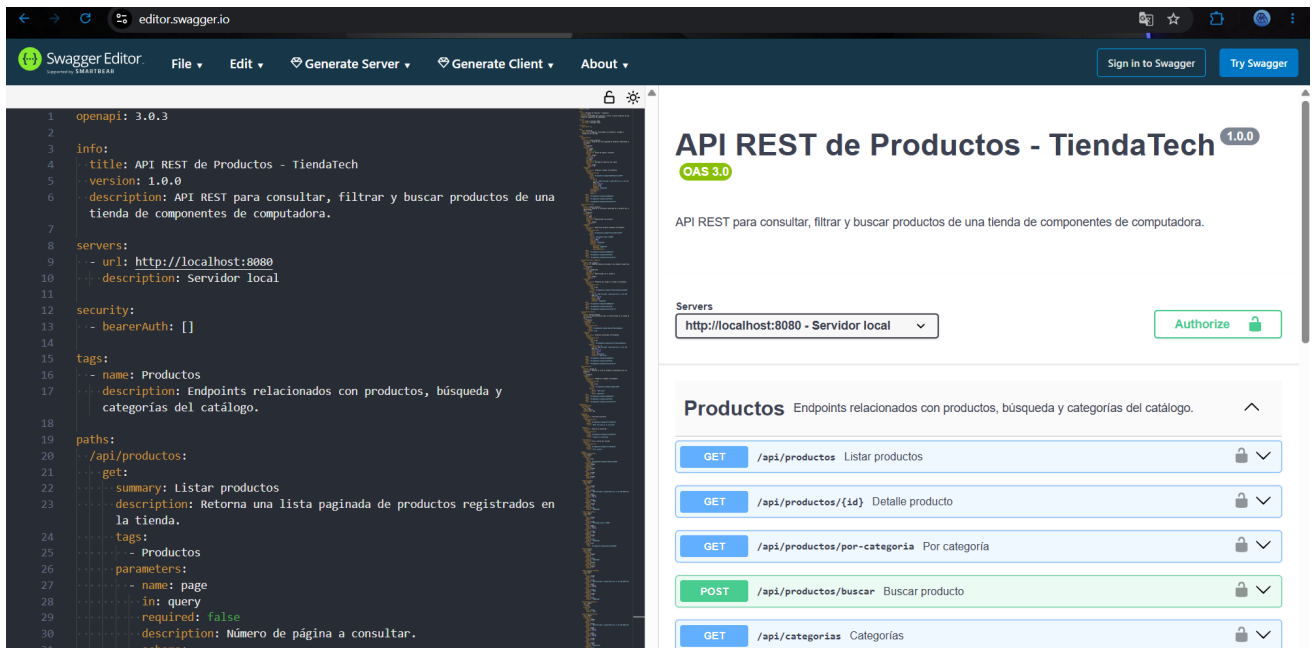


Figure 1: Contrato OpenAPI validado en Swagger Editor.

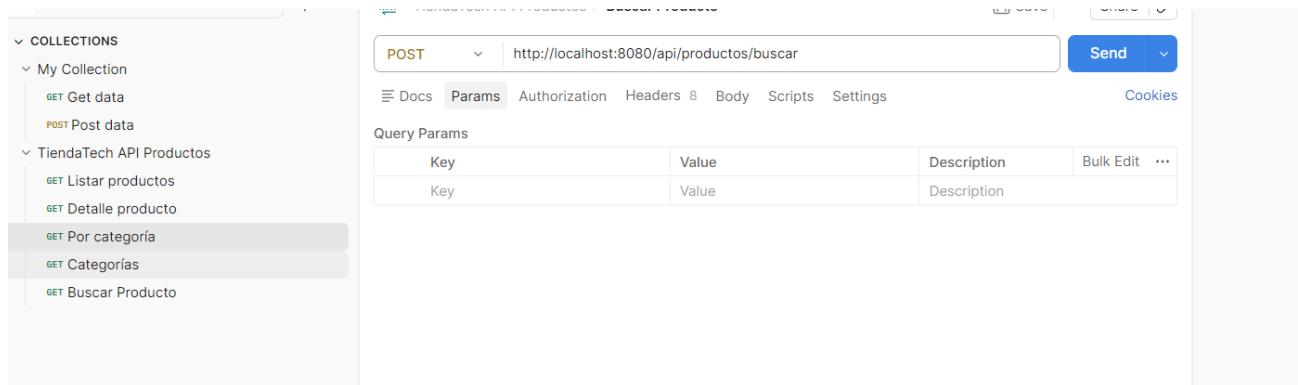


Figure 2: Colección de endpoints documentados en Postman.

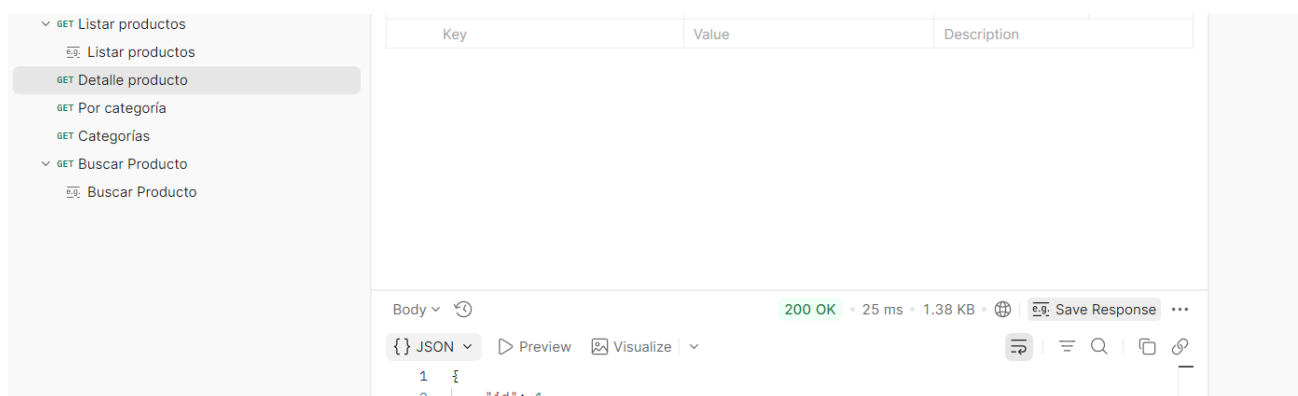


Figure 3: Ejecución exitosa de endpoint con respuesta HTTP 200 OK.